Part Three

Tyler Dinh[1], Kevin Tang[1], and Demi Babalola[1]

[1]Washington University in St. Louis

October 15, 2026

1 Algorithmic Design and Implementation Details

The objective function is Mean Squared Error (MSE), defined as $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$.

Then, the gradient equation used for parameter updates for step i can be calculated as:

$$w^{i+1} = w^i - \alpha\frac{\delta MSE}{\delta m}$$

$$\frac{\delta MSE}{\delta m} = \frac{-2}{n}\sum_{i=1}^{n}(y_i - (mx_i + b))$$

Algorithm 1 Custom Gradient Descent Function

procedure GRADIENT DESCENT(W^0, b^0, X, y, η, N)
  for $i = 0$ to N **do**
    $\text{err} = f(X, W^i, b^i) - y$
    $W^{i+1} = W^i - \eta \cdot \frac{2}{|X|} X^\top \text{err}$
    $b^{i+1} = b^i - \eta \cdot 2\,\overline{\text{err}}$
  end for
  return W^N, b^N
end procedure

Our dependencies were:

ucimlrepo 0.0.7, pandas 3.0.5, matplotlib 3.11.2, numpy 2.5.3, scikit-learn 1.9.1, statsmodels 0.15.0, python 3.12

We designated rows 501–630 as the testing set (130 samples) and the remaining 900 samples as the training set, and split into a features matrix and output vector for each set. Model parameters were not updated based on the test dataset.

We set a list of learning rates for α = [1e-9, 1e-8, 1e-7], initial coefficient weights matrix to be all zeroes, initial bias to be zero, and iterations to be 1000. We stop the algorithm after i=1000 completes.

Then, loop over lrs and find the coefficient weights matrix, bias, along with mse and r2 over time as requested.

2 Feature Scaling and Gradient Descent Dynamics

The two implementations for the unscaled and scaled features for the multivariate model used the same features, the same train/test split, the same zero-initialized weights, the same 1000 iterations, and the same gradient descent functions. In the implementation for the unscaled features multivariate model, raw features are used directly, and since their magnitudes vary quite wildly, the loss surface becomes quite sharp. As such, we chose to use extremely tiny learning rates ($1e-9$ to $1e-7$) so that the curve would converge. In the implementation for the scaled features multivariate model, the features are standardized using the training set's mean and standard deviation, which makes the loss function much smoother. As a result, we use much larger learning rates ($1e-4$ to $1e-2$) which lead to convergence of gradient descent more smoothly.

Our implementation of multivariate model using untransformed features, and another using scaled features, until they both achieve a positive r2

Chart here from assets/unscaled vs scaled

Feature scaling (normalization by z-score) allowed the weights of the model to converge more quickly. Because the values in general are smaller than without scaling, we tested a higher-valued set of learning rates. When we used the smaller learning rates we had to use for unnormalized data, the model did not learn at a sufficient pace or converge. Also, scaling brings the values from different features closer to each other, so training using scaled features becomes smoother as well, as shown in the above chart. Overall, we find that our algorithm performs better on the test set (lower MSE and higher R2) when we scale the features.

A linear transformation does not meaningfully change the optimal mathematical solution. Although the weights are different for a scaled model, the output of the scaled model on scaled inputs is identical to the output of the unscaled model on unscaled inputs. We can see that the MSE on the test set for both a scaled and unscaled model are identical, at 141.25799330049728 and 0.35125143823506555 respectively.

3 Linear vs. Non-Linear Feature Transformations

chart here from assets/nonlinear

The log transformation increases convergence speed and stability greatly over the unscaled features, and we were able to use much higher learning rates without the model diverging. This is expected, because similarly to the z-score scaling, it brings the distribution of all the features closer to one another by bringing down extremely large values so that gradient descent proceeds more smoothly.

table here from assets/nonlinear table

While the untransformed and normalized models perform the same on the testing and training sets, the log transform, which is nonlinear, improved the optimal solution and overall predictive performance. This is because the log transformation helps with features that don't have a linear relationship with the target feature, as well as making right-skewed distributions more normal by stretching out low-valued data and compressing high-valued data, which reduces heteroscedasticity.

4 Univariate vs. Multivariate Model Dyanamics

The easiest model to train was Concrete, as it quickly converged to a positive R2 during training with the learning rates we tested. The most difficult models were Superplasticizer, Coarse Aggregate, and Fine Aggregate, which we were unable to get to converge properly at the learning rates that worked, and if we increased the learning rates too much, the model diverged and gave us an overflow error/nan.

assets/coeff table here

The coefficients for the models involving individual features are different from the corresponding coefficient in the multivariate model. For example, perhaps the largest differences are that Fly Ash, Coarse

Aggregate, and Fine Aggregate have a positive correlation with the target feature in the multivariate model while they have a negative correlation in their respective univariate models.

Univariate coefficients are not equal to multivariate coefficients because some features in our dataset may correlate with one another. Therefore, when training the univariate model for a single feature such as Fly Ash, the single coefficient will also account for the effects of all other features that are correlated to Fly Ash, which is omitted variable bias. When we train the multivariate model however, weights will be assigned based on only the contribution of that feature to achieve the lowest possible error, since the overall model now includes all features, eliminating omitted variable bias.

5 Statistical Inference and Regression Analysis

/assets p vals table

There are trends between which predictors are important, and which univariate models were easier to train? For example, a lower p-value meant the univariate model corresponding to that feature was easier to train. We found that Cement was the univariate models that converged the most quickly and smoothly, agreeing with its small p-values, while we had some more trouble getting Superplasticizer, Coarse Aggregate, and Fine Aggregate to converge properly at the learning rates we tried, which makes sense, looking at their relatively large p-values.

We observe log transformation changes the p-values significantly, while the p-values of the normalized model are exactly the same as without feature scaling. This is because normalization is a linear transformation, so it will not affect the underlying shape of the distribution of any of the features, and the t-statistic remains the same as before mathematically. However, with log scaling, it is not linear, and the shapes of the distributions are not preserved so the p-value can change, decreasing especially in instances where the original data was not linear/skewed and made more linear by the transformation.

6 Code and Reproducibility Access